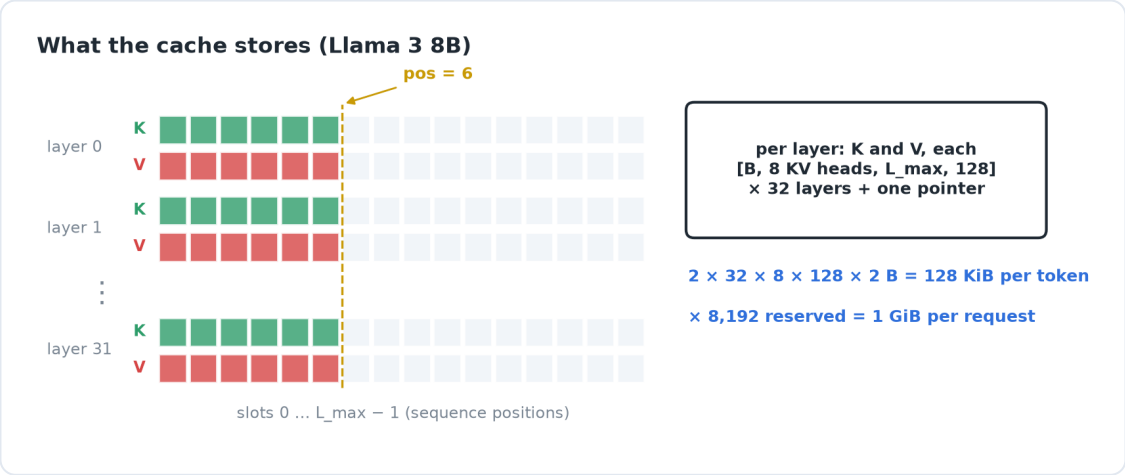
17 · Build a KV Cache

Source: fanout.sh / inference-eng / 03-04 · ~7 min

> **IN SHORT**
>
> A KV cache is **one key tensor and one value tensor per layer**, shaped **[batch, 8 KV heads, max length, 128]** for Llama 3 8B, plus a **pointer** saying how many slots are filled: **128 KiB per token**, so 8,192 reserved tokens is exactly **1 GiB per request**. **Preallocate** the buffer and write each new K/V **in place** (concatenating copies everything every step: 33.6M token copies over 8k tokens). Prefill writes slots 0 … P−1 in one shot; each decode step writes at the pointer, reads only slots 0 … pointer, and advances it. Four classic bugs: **wrong RoPE position, reading empty slots, not resetting, copying KV heads**. Prove it with identical greedy tokens, matching byte counts and flat step time. The hidden cost: reserved but **unused memory**.

1. What we store

In every layer, attention produces a key and a value **per token, per KV head**. So each layer gets two tensors:



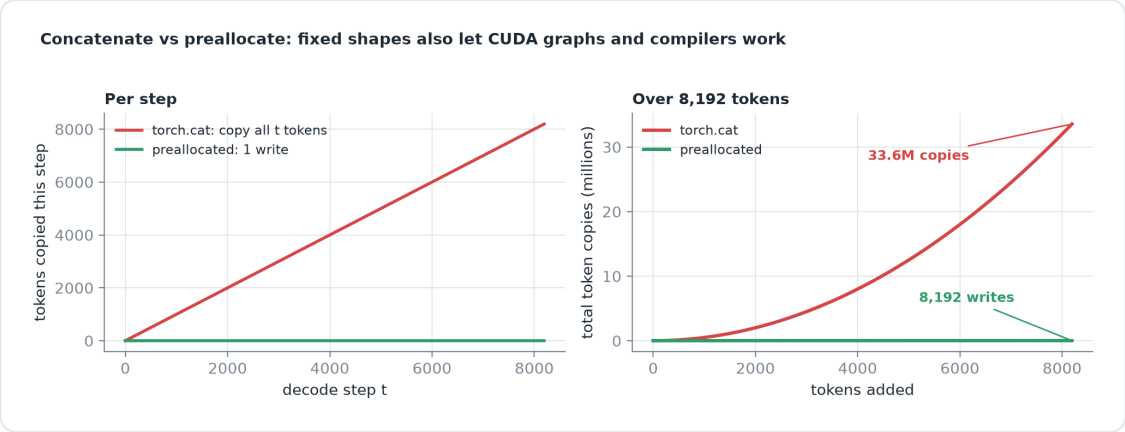
32 layers × (K, V), each [B, 8, L_max, 128], plus one small but essential piece: the position pointer.

The per-token size is lesson 05's 128 KiB (2 × 32 × 8 × 128 × 2 bytes). Reserve 8,192 tokens and a single request's cache is **exactly 1 GiB**.

2. How the cache grows

```
# naive: a brand-new tensor every step, copying everything
k = torch.cat([k, k_new], dim=2)

# better: allocate once, write into the next empty slot
k_cache = torch.empty(B, 8, L_max, 128)
k_cache[:, :, pos] = k_new
pos += 1
```

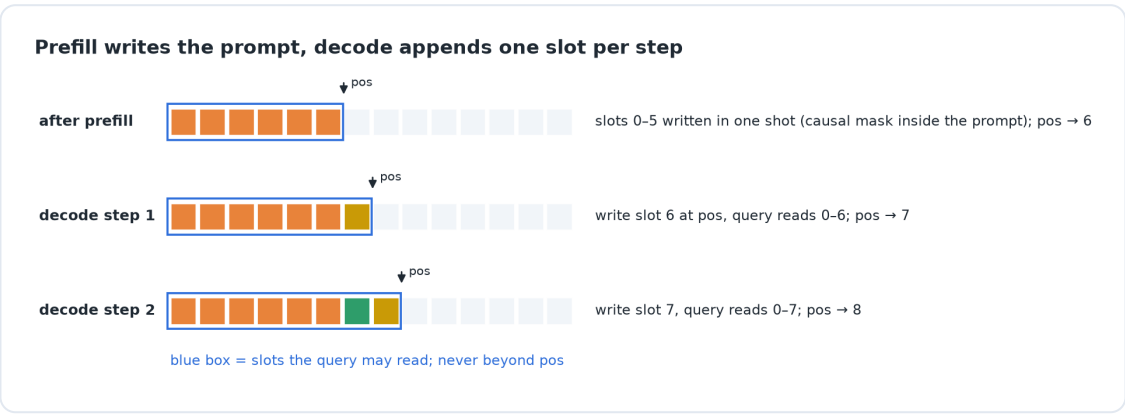


Concatenation: more than 33 million token copies just to add 8,192 tokens. Preallocation: one small write per step, whatever the length.

INTUITION

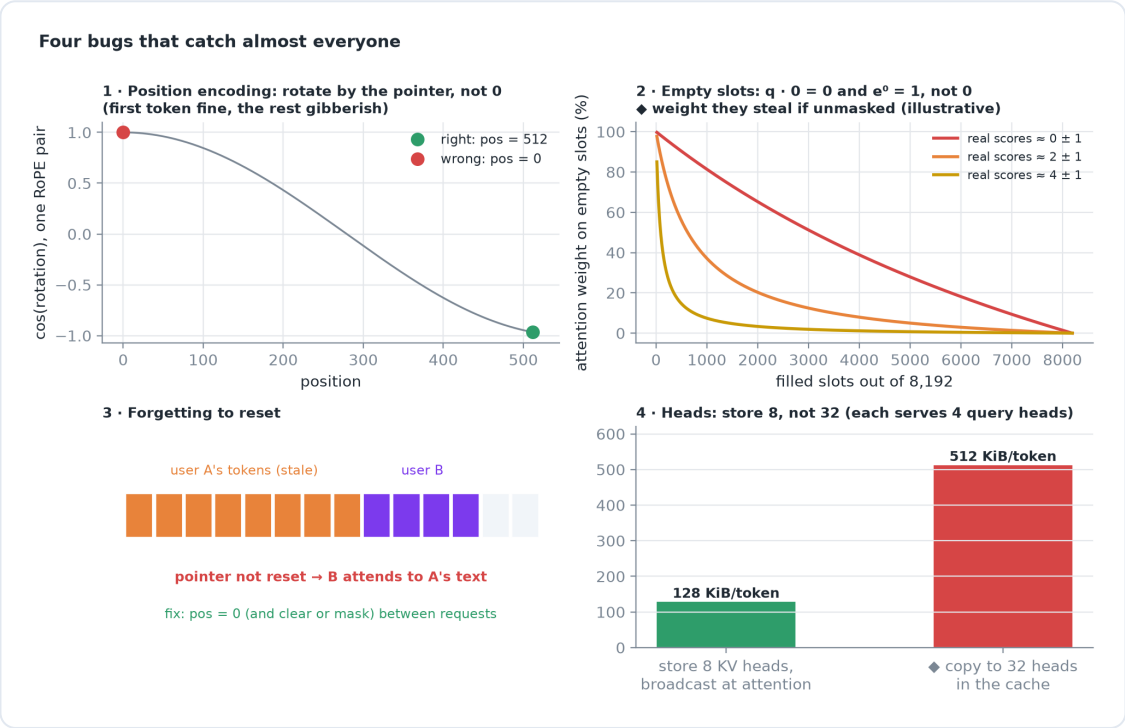
With preallocation the **tensor shapes never change**, exactly what **CUDA graphs and compilers** need. Production code (e.g. torchtune) uses a fixed buffer plus a position counter.

3. Prefill writes, decode appends



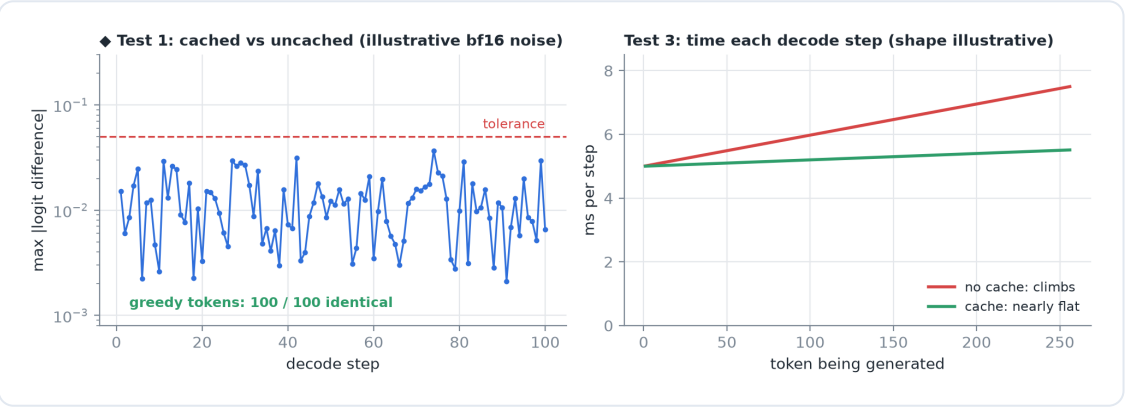
- **Prefill:** the whole prompt arrives at once; its K/V go into slots **0 … P−1 in one shot**, in every layer. Inside the prompt, attention still uses the **causal mask**. The pointer jumps to the end of the prompt.
- **Decode:** each step writes the new token's K/V **at the pointer**, its query reads slots **0 … pointer and nothing beyond**, then the pointer moves forward by one.

4. Four bugs that catch almost everyone



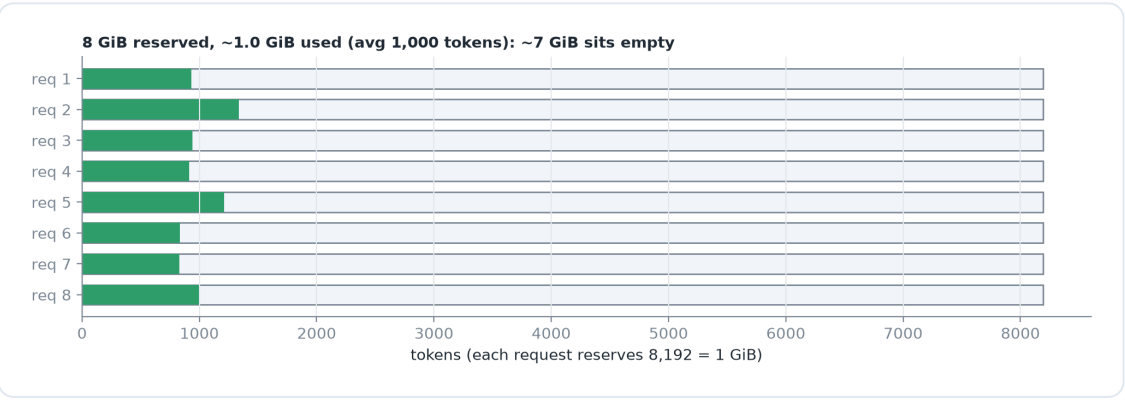
1. **Position encoding.** Llama rotates each query and key by an angle set by the token's **position** (RoPE). In decode you feed one token, so it's tempting to call it position 0, but it's really **position 512** (or wherever the pointer is). Get it wrong and the first token is fine, then **gibberish**.
2. **Reading empty slots.** A preallocated buffer is full of zeros. A zero key gives score **0**, and **$e^0 = 1$, not 0**, so unfilled slots quietly **steal attention weight**. Always **slice or mask** to the filled length.
3. **Forgetting to reset.** If the pointer isn't cleared between requests, the next user's prompt **attends to the previous user's text**.
4. **Heads.** Llama stores only **8 KV heads**, each shared by 4 query heads. **Broadcast** them at attention time; don't copy them into the cache.

5. Prove it works



- **Correctness:** generate 100 tokens greedily with and without the cache; the token sequences must **match exactly**. Caching changes **how** we compute, not **what**. Logits won't be bit-identical in 16-bit, but should agree **within a small tolerance**.
- **Memory:** allocated bytes must equal 2 × layers × KV heads × head dim × length × bytes per number.
- **Speed:** time each decode step: **flat with the cache**, climbing without.

6. The hidden cost of preallocating



Eight requests reserved for 8,192 tokens each set aside 8 GiB. If they average 1,000 tokens, about 7 GiB sits empty, which limits how many requests fit on the GPU. Paged memory fixes this.

7. Recap

Lesson 17 on one card

SHAPE

per layer: K and V [B, 8, L_max, 128]
× 32 layers + a position pointer
128 KiB/token, 1 GiB per 8k request

GROW

preallocate once, write at pos
not torch.cat (33.6M copies)
fixed shapes: CUDA graphs, compile

FOUR BUGS

RoPE at the true position
mask or slice empty slots ($e^0 = 1$)
reset per request; store 8 KV heads

PROVE IT

greedy tokens identical with/without
logits within tolerance; bytes match
step time flat. Cost: reserved waste